Universidad Argentina de la Empresa
Facultad de Ingeniería y Ciencias Exactas

Desarrollo de Aplicaciones I · Trabajo Práctico Obligatorio

take a break

Soltá el celu. Ganás vos.

Preentrega de análisis y diseño

Integrantes Papa, Valentin · LU 1197189
Velazquez, Joaquin Eduardo · LU 1186723
Jerez, Juan · LU 1152497
Maci, Abril · LU 1152146
Docente Peña, Alejandro Francisco (titular)
Narducci, Adrian Alberto (ayudante)
Fecha 8 de octubre de 2026

Índice

1. Introducción **3**
1.1. Cómo está organizado este documento . . . 3

2. Empatizar: el problema y quién lo tiene **3**
2.1. Descripción del problema . . . 3
2.2. Usuario principal . . . 4
2.3. Contexto de uso . . . 4
2.4. Recorrido actual del usuario . . . 5

3. Definir: punto de vista y principios **5**
3.1. Punto de vista . . . 5
3.2. ¿Cómo podríamos...? . . . 5
3.3. Principios de diseño . . . 5

4. Idear: la propuesta **6**
4.1. Propuesta de solución . . . 6
4.2. ¿Por qué una app móvil y no una web? . . . 6
4.3. Caso de negocio y propuesta de valor . . . 6
4.4. Ideas evaluadas y descartadas . . . 7
4.5. Alcance . . . 7

5. Requisitos **8**
5.1. Requisitos funcionales . . . 8
5.2. Reglas de validación . . . 8
5.3. Requisitos no funcionales . . . 10

6. Prototipar: diseño de la experiencia **10**
6.1. Del boceto a Figma . . . 10
6.2. Look & feel . . . 11
6.3. Ley de Fitts y fricción . . . 11
6.4. Pantallas principales . . . 12
6.4.1. Componentes y jerarquía . . . 13
6.4.2. Estados . . . 14
6.5. Flujo de pantallas . . . 14

7. Testear: plan de validación **16**

8. Arquitectura **16**

8.1. Capas y responsabilidades 16
8.2. Flujo de datos 17
8.3. Cómo se detecta una pausa 17
8.4. Topología de infraestructura 18

9. Persistencia y datos **19**

10. Estrategia Offline First **19**

11. Tecnologías previstas **20**

12. Repositorio y equipo **20**
12.1. Repositorios 21
12.2. Forma de trabajo 21
12.3. Integrantes, roles y responsabilidades 21

A. Todas las pantallas **23**

B. Sistema de diseño **28**

C. Bocetos originales **28**

1. Introducción

Nombre provisorio: Take a Break.

Take a Break es una app para Android que convierte el tiempo que una persona pasa lejos del celular en puntos, y esos puntos se canjean por premios en comercios del barrio. No hay que activar nada. La app se entera sola de que el teléfono estuvo quieto y, si la pausa cumple las reglas, suma.

Problema → Usuario → Solución → Valor

Las herramientas para usar menos el celular miden o bloquean, pero no dan nada a cambio → adultos jóvenes que ya lo intentaron y abandonaron → detección pasiva de pausas y premios reales cerca → un hábito más sano para el usuario y clientes nuevos para el comercio.

1.1. Cómo está organizado este documento

Trabajamos con design thinking, así que el documento sigue sus cinco etapas. La tabla muestra dónde está cada punto que pide la consigna.

Etapa	Puntos del entregable	Sección
Empatizar	2. Descripción del problema · 3. Usuarios · 4. Contexto de uso	2
Definir	Punto de vista, preguntas guía y principios de diseño	3
Idear	1. Nombre · 5. Propuesta · 6. Por qué móvil · 7. Caso de negocio · 8. Alcance	4
Requisitos	9. Requisitos funcionales · 10. Requisitos no funcionales	5
Prototipar	11. Diseño en Figma · 12. Flujo de pantallas	6
Testear	Plan de validación con usuarios	7
Construir	13. Arquitectura · 14. Offline first · 15. Tecnologías	8 a 11
Equipo	16. Repositorio · 17. Integrantes y roles	12

El diseño está publicado en Figma (`https://www.figma.com/community/file/1687633708033076651/take-a-break-diseno`) y los archivos fuente están en el repositorio `docs`, carpeta `figma/`. Todas las pantallas aparecen en el Anexo A.

2. Empatizar: el problema y quién lo tiene

2.1. Descripción del problema

¿Qué problema existe?
Mucha gente quiere usar menos el celular y no lo logra sostener. Las herramientas que existen muestran estadísticas o bloquean apps, pero no le dan nada a la persona cuando lo consigue.

¿Quién lo tiene?
Adultos de 18 a 35 años que usan el teléfono todo el día y que ya probaron alguna app de tiempo de pantalla.

¿En qué contexto ocurre?
En la vida diaria: en casa, en el trabajo, en el colectivo. El celular siempre está a mano y las notificaciones compiten por la atención.

¿Cómo se resuelve hoy?
Con Bienestar digital de Android o Tiempo en pantalla de iOS, y con apps como Forest o one sec, que obligan a iniciar una sesión o agregan una espera antes de abrir otras apps.

¿Qué dificultades tiene esa solución?
Piden un esfuerzo activo (acordarse de iniciar la sesión, configurar límites), funcionan como castigo y el logro no tiene ningún efecto fuera del teléfono. Al tiempo se ignoran o se desinstalan.

¿Por qué una app móvil mejoraría la situación?
El dato que importa, cuánto tiempo estuvo el teléfono sin usarse, lo registra el propio sistema operativo. Solo una app instalada en el teléfono puede leerlo sin pedirle nada al usuario.

2.2. Usuario principal

Atributo	Descripción
Características	Entre 18 y 35 años, usa el celular muchas horas por día, quiere bajar ese uso pero no lo sostiene.
Necesidad principal	Un motivo concreto para dejar el teléfono, no más información sobre cuánto lo usa.
Contexto de uso	Casi todo el uso es pasivo: la app trabaja en segundo plano. Se abre un rato para ver puntos o canjear.
Frecuencia	Entre 2 y 4 aperturas breves por día. Un canje por semana, aproximadamente.
Restricciones	Poco tiempo, una sola mano, a veces caminando o parado frente al mostrador de un local.
Conectividad	Intermitente. Durante la pausa el teléfono puede estar en modo avión o sin señal.
Condiciones situacionales	Luz variable al mostrar el código en un local; apuro porque hay gente esperando.
Dificultades actuales	Se olvida de activar las herramientas y pierde la motivación porque no recibe nada a cambio.

Persona: Martina, 27 años

Trabaja como diseñadora en Palermo y vive a diez cuadras de la oficina. Revisa el celular apenas se despierta y otra vez antes de dormir. Probó Forest dos semanas y lo dejó: se olvidaba de plantar el árbol. Le gustaría leer más y salir a caminar, y le encantan los cafés de especialidad del barrio.
"Sé que lo uso demasiado. Lo que no tengo es una razón para dejarlo."

La persona es un arquetipo armado con supuestos del equipo. La idea es validarla con las entrevistas del plan de la sección 7.

El comercio adherido es un usuario secundario. Publica premios y los entrega en el local, pero en esta versión no tiene una app propia: sus datos se cargan como datos semilla.

2.3. Contexto de uso

Seguimos la idea de computación ubicua de Mark Weiser: la mejor interacción con esta app es no tener que abrirla. El usuario nunca declara que empieza una pausa. Deja el teléfono y la app se ocupa del resto. Solo vuelve a aparecer para avisar que sumó puntos.

Hay dos momentos en los que sí hay interacción, y los dos son cortos:

- **Mirar cómo va**: abre la app desde la notificación, ve el saldo y cuánto le falta para su meta. Menos de 10 segundos.
- **Canjear**: está en el local, con una mano, quizás con gente atrás. Necesita encontrar el premio, confirmar y mostrar un código que se lea bien.

2.4. Recorrido actual del usuario

Así se ve hoy el intento de usar menos el celular con las herramientas que existen. La última columna es lo que tomamos para el diseño.

Etapa	Qué hace	Cómo se siente	Oportunidad
Decide	"Esta semana lo uso menos."	Motivada	Aprovechar el impulso sin pedir configuración
Configura	Instala una app, arma límites y horarios	Cansada	Cero configuración
Usa	Tiene que acordarse de activar el modo foco	Se olvida	Detectar las pausas sola
Ve el resultado	Un gráfico o un árbol virtual	Indiferente	Un premio real y cerca
Abandona	Desinstala o ignora los avisos	Culpa	Reglas amables, racha y meta

3. Definir: punto de vista y principios

3.1. Punto de vista

Martina, que quiere usar menos el celular pero abandona las apps que lo intentan, **necesita** que desconectarse le dé algo concreto a cambio sin tener que acordarse de nada, **porque** las herramientas actuales le piden esfuerzo y solo le devuelven estadísticas.

3.2. ¿Cómo podríamos...?

HMW1 ¿Cómo podríamos registrar una pausa sin que el usuario haga nada?

HMW2 ¿Cómo podríamos lograr que una hora sin el celular valga algo fuera de la pantalla?

HMW3 ¿Cómo podríamos evitar que alguien sume puntos durmiendo, sin que la regla se sienta injusta?

HMW4 ¿Cómo podríamos canjear un premio en el mostrador en segundos?

3.3. Principios de diseño

De esas preguntas salen cinco principios. Los usamos para decidir cada pantalla y cada requisito.

	Principio	Qué implica
P1	Cero toques para lo central	La pausa se detecta sola. El usuario no inicia ni termina nada.
P2	La moneda es el tiempo	1 minuto de pausa es 1 punto. Los precios también se muestran en horas (“te falta 1 h”).
P3	Una acción, al alcance del pulgar	Una sola acción principal por pantalla, grande y abajo (ley de Fitts).
P4	Reglas a la vista	Si una pausa no suma, la app dice por qué. Nada de reglas ocultas.
P5	Sin red también anda	Detectar, sumar y consultar funcionan offline. Canjear necesita conexión y la app lo explica.

4. Idear: la propuesta

4.1. Propuesta de solución

Nombre provisorio	Take a Break
Descripción	App Android que detecta sola los períodos en que el teléfono estuvo bloqueado, los valida con reglas simples y acredita puntos canjeables en comercios cercanos.
Usuario principal	Adultos jóvenes que quieren usar menos el celular (sección 2).
Propuesta de valor	Desconectarse deja de ser un castigo y pasa a tener premio.
Escenario principal	Martina deja el teléfono en la mesa durante la cena. Cuando lo agarra le llega “Break validado: +60 pts. Ya te alcanza para un café”. Al día siguiente pasa por el café, canjea y muestra el código en el mostrador.
Beneficio esperado	Más tiempo sin pantalla para el usuario y clientes nuevos para el comercio.

4.2. ¿Por qué una app móvil y no una web?

- **El dato está en el teléfono.** Los eventos de pantalla y bloqueo los registra Android (`UsageStatsManager`). Un sitio web no puede leerlos.
- **Trabaja sin que la abran.** WorkManager consulta esos eventos en segundo plano, cada 15 minutos, sin un proceso corriendo todo el tiempo.
- **Avisa en el momento justo.** Una notificación local cierra el ciclo de recompensa apenas termina la pausa, aunque no haya red.
- **Acompaña al usuario al local.** El mapa usa la ubicación del teléfono y el código se muestra en el mostrador.

4.3. Caso de negocio y propuesta de valor

El proyecto genera valor para dos partes:

- **Usuario** (valor social y de experiencia): un incentivo positivo para desconectarse, gratis.
- **Comercio local** (valor económico): una forma barata de atraer gente nueva, pensada sobre todo para locales chicos o recién abiertos.

La idea se inspira en Pasito, que premia los pasos con canjes en comercios. En nuestro caso el comercio paga una suscripción mensual para publicar sus premios y esa suscripción financia la

plataforma. El usuario nunca paga. Cada canje lleva a una persona al local, que muchas veces compra algo más.

En esta versión no hay pagos ni suscripciones reales. Los comercios y sus premios se cargan como datos semilla en el backend del equipo.

4.4. Ideas evaluadas y descartadas

Antes de llegar a la propuesta comparamos varias alternativas contra los principios de la sección 3.

Idea	**Por qué no**
Bloquear apps o agregar una espera	Es el enfoque de castigo que el usuario ya abandonó.
Botón "empezar pausa"	Depende de la memoria del usuario. Va contra P1.
Servicio escuchando `SCREEN_OFF`	Necesita un proceso vivo todo el día, gasta batería y muestra una notificación fija. Lo reemplaza la consulta periódica a `UsageStatsManager`.
Acelerómetro para confirmar que el celular está quieto	No aporta información que los eventos de bloqueo no den, y consume batería.
Ubicación en segundo plano y geofencing	Es un permiso sensible y el caso central no lo necesita.
Premios virtuales (insignias, árboles)	No resuelven la falta de un incentivo tangible.

4.5. Alcance

Priorizamos cerrar un solo ciclo completo: **detectar** → **validar** → **premiar** → **ubicar** → **canjear**.

Dentro del alcance

- Detección pasiva de pausas y validación con reglas anti-abuso.
- Saldo, meta, racha e historial, todo offline.
- Catálogo de 5 o 6 comercios de ejemplo, con mapa y lista.
- Canje con código y confirmación de la entrega con el PIN del comercio.
- Notificación local al validar una pausa.
- Acceso con Google o email.

Fuera del alcance

- App o panel para que los comercios carguen premios.
- Pagos y suscripciones reales.
- Navegación hasta el local y geofencing.
- Ubicación en segundo plano.
- Desafíos grupales y ranking.
- Antifraude del lado del servidor (ver limitaciones).

Limitación conocida. Las pausas se detectan y se validan en el teléfono, y nunca llegan al servidor. El servidor lleva un libro de movimientos por usuario, con el total de puntos de cada día, los débitos de los canjes y las devoluciones, y a partir de ese libro define el saldo que se puede gastar. Una app modificada podría informar puntos que no ganó, pero como máximo 240 por día. Alcanza para un prototipo con comercios de prueba. Un producto real necesitaría una forma de verificar las pausas del lado del servidor.

5. Requisitos

5.1. Requisitos funcionales

En todos los casos el usuario involucrado es el usuario final de la app.

RF01. Detectar y validar pausas

Descripción: la app detecta sola los períodos en que el teléfono estuvo bloqueado, los guarda y les aplica las reglas de validación de la tabla 1.
Criterio de aceptación: si el teléfono queda bloqueado 60 minutos un día hábil entre las 9 y las 23 h, la pausa aparece como válida con +60 puntos y llega una notificación, a más tardar 30 minutos después de desbloquearlo o apenas se abre la app. Funciona igual en modo avión.

RF02. Consultar saldo, meta e historial

Descripción: el usuario ve su saldo, el avance hacia la meta elegida, la racha y el historial de pausas.
Criterio de aceptación: sin conexión, Inicio y Actividad muestran saldo, meta, racha y las pausas de los últimos 30 días. Cada pausa que no sumó indica el motivo.

RF03. Explorar premios cercanos

Descripción: el usuario ve en un mapa y en una lista los premios de los comercios adheridos, y puede filtrar los que ya le alcanzan.
Criterio de aceptación: con permiso de ubicación la lista se ordena por distancia. Sin permiso se muestra ordenada de la A a la Z, sin bloquear la pantalla. Sin conexión se muestra el catálogo guardado con la hora de la última actualización.

RF04. Canjear un premio

Descripción: el usuario confirma el canje manteniendo presionado el botón. El servidor valida el stock y el saldo, descuenta los puntos y genera un código para mostrar en el local. El cajero confirma la entrega escribiendo el PIN del comercio en el teléfono del usuario.
Criterio de aceptación: con conexión, el código aparece en menos de 3 segundos y vence a los 10 minutos; si vence sin usarse, los puntos vuelven al saldo. Sin conexión no se canjea y no se descuentan puntos. Con el PIN correcto, el canje pasa a usado en un único cambio de estado atómico, así un código no se puede usar dos veces aunque lleguen dos pedidos juntos. Al quinto PIN incorrecto, el canje queda rechazado y los puntos vuelven al saldo.

5.2. Reglas de validación

Las reglas buscan premiar la desconexión elegida y no el tiempo en que el teléfono quedó quieto por otros motivos.

Regla	**Valor**	**Motivo**
Puntos	1 por minuto válido	El usuario no tiene que hacer conversiones (P2).
Duración mínima	30 min de lunes a viernes, 45 min el fin de semana	Una pausa corta no cambia el hábito. El fin de semana es más fácil estar inactivo sin decidirlo.
Horario de descanso	De 23 a 9 h no suma	Evita sumar puntos durmiendo. La pausa queda igual en el historial.
Pausa que cruza las 23 h	Suman solo los minutos fuera del horario de descanso	Evita cortar la pausa de golpe.
Tope diario	240 puntos	Limita la acumulación con muchas pausas seguidas.
Fin de la pausa	Al desbloquear, no al encender la pantalla	Una notificación enciende la pantalla, pero eso no es usar el teléfono.

Tabla 1: Reglas de validación de pausas.

5.3. Requisitos no funcionales

ID	Tipo	Requisito	Cómo se verifica
RNF01	Batería	Sin servicios en primer plano ni procesos permanentes. La detección corre en WorkManager cada 15 min. Consumo atribuido a la app menor al 1 % en 24 h.	Revisión del manifest y medición con Battery Historian.
RNF02	Offline	Detectar pausas, ver saldo e historial y abrir el catálogo guardado funcionan en modo avión.	Prueba manual en modo avión.
RNF03	Privacidad	Solo se leen eventos de pantalla y bloqueo. No se guardan ni se envían nombres de apps. La ubicación se pide en primer plano y no sale del teléfono.	Revisión de código. El manifest no declara `ACCESS_BACKGROUND_LOCATION`.
RNF04	Confiabilidad	Una pausa se guarda en Room antes de notificar. Cerrar la app a la fuerza no pierde datos y la misma pausa nunca se cuenta dos veces.	Test instrumentado y test unitario que guarda dos veces la misma pausa.
RNF05	Usabilidad	Canjear un premio desde Inicio lleva como máximo 2 toques y 1 gesto de mantener. Objetivos táctiles de 48 dp o más.	Prueba con usuarios y Accessibility Scanner.
RNF06	Accesibilidad	Contraste AA, descripción en todos los íconos, uso completo con TalkBack y con fuente al 200 %.	Accessibility Scanner y prueba manual.
RNF07	Rendimiento	Arranque en frío menor a 2 s en un equipo de gama media.	Macrobenchmark.
RNF08	Compatibilidad	Android 9 (API 28) en adelante, porque los eventos de bloqueo existen desde esa versión.	Emuladores API 28 y 35.
RNF09	Mantenibilidad	Las reglas de validación están en el dominio, en Kotlin puro y sin dependencias de Android. Cobertura de tests del paquete `domain` de 80 % o más.	Reporte de cobertura (Kover).
RNF10	Seguridad	Tráfico solo por HTTPS. El token de sesión se guarda cifrado con Android Keystore.	Configuración de red sin *cleartext* y revisión de código.

6. Prototipar: diseño de la experiencia

6.1. Del boceto a Figma

El archivo de Figma se puede ver en `https://www.figma.com/community/file/1687633708033076651/take-a-break-diseno`.

Empezamos con bocetos en papel (Anexo C). Al pasarlos a Figma mantuvimos la ecuación “1 hora = 1 recompensa” del onboarding, las tarjetas de premios y la barra inferior de tres íconos. Cambiamos cuatro cosas:

- **Acceso unificado.** Los botones “Iniciar sesión”, “Google” y “Sign up” quedaron en dos: “Continuar con Google” y “Continuar con email”. Con menos opciones se decide más rápido (ley de Hick).
- **“Premios cerca” pasó a ser “Te alcanza hoy”.** En Inicio solo aparece lo que el usuario ya puede canjear.
- **Meta con barra de progreso.** El boceto pedía “una barra para saber cuánto falta”. Ahora es una tarjeta de meta que muestra lo que falta en puntos y en horas.
- **El gráfico se mudó a Actividad.** Inicio queda para lo que se consulta a diario.

6.2. Look & feel

Tomamos como referencia el estilo de Pasito: minimalista, con un verde bosque de marca, lima para los puntos y los botones sobre fondo oscuro, y fondo crema. Botones y chips tienen forma de píldora y no hay sombras: la jerarquía sale del color y del tamaño. Los títulos van en Bricolage Grotesque y el texto en DM Sans. El sistema completo está en el Anexo B.

6.3. Ley de Fitts y fricción

La ley de Fitts dice que el tiempo para tocar algo crece con la distancia y baja con el tamaño del objetivo. En un teléfono que se usa con una mano, la distancia se mide desde el pulgar. Por eso:

- La acción principal de cada pantalla mide 56 dp de alto por 320 dp de ancho y siempre está abajo.
- La barra inferior tiene solo tres destinos, de 120 dp cada uno.
- Lo que se usa poco, como el perfil, va arriba.
- Canjear no se puede deshacer, así que se confirma manteniendo presionado un botón de 64 dp. Evita toques accidentales sin agregar un diálogo.



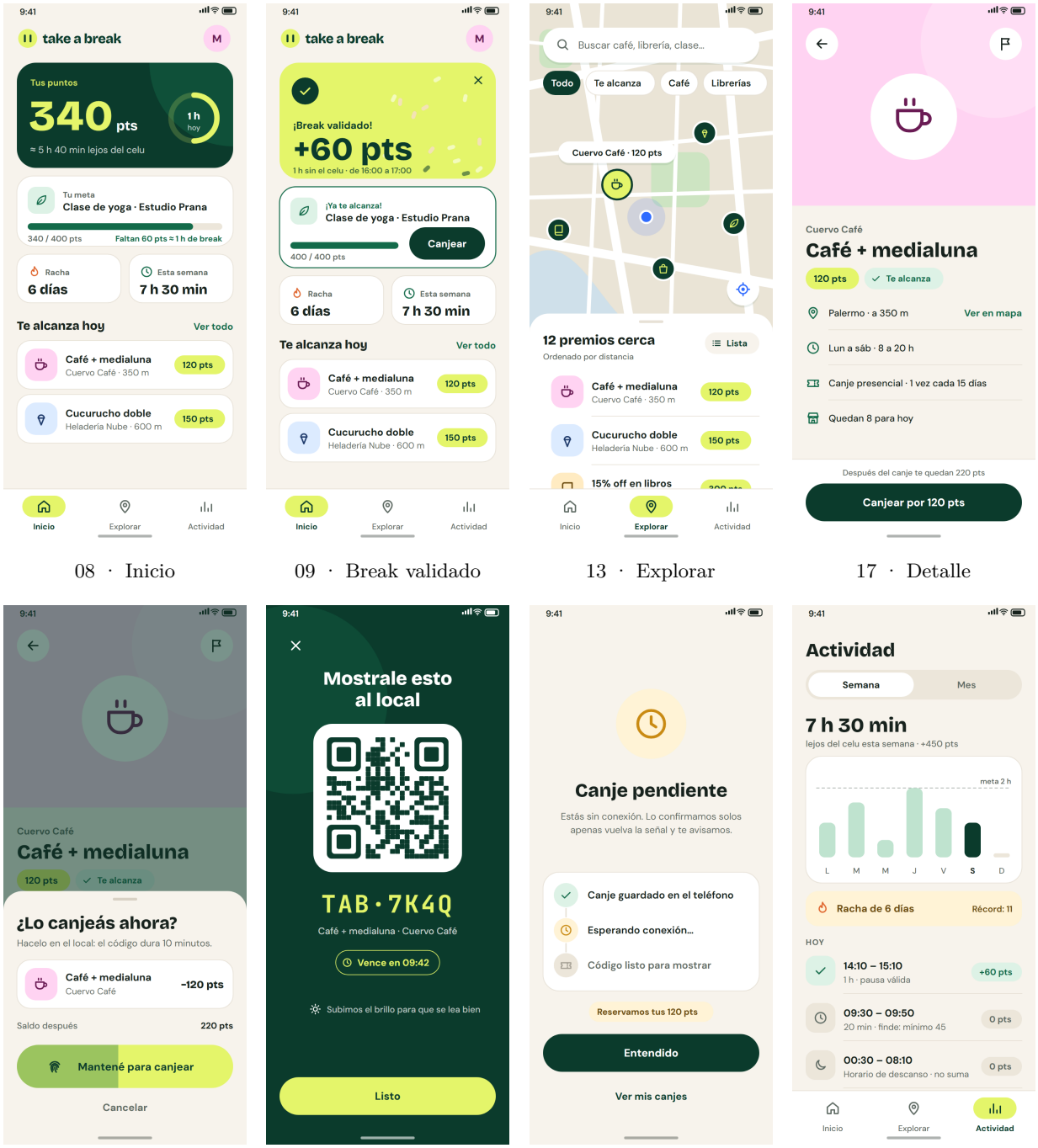
Figura 1: Zona del pulgar sobre Inicio y la confirmación de canje. En rojo, los objetivos táctiles.

Medimos la fricción contando interacciones por tarea:

Tarea	**Interacciones**	**Cómo**
Registrar una pausa	0	Detección pasiva.
Ver cuánto sumó	1 toque	Tocar la notificación.
Canjear un premio que alcanza	2 toques y mantener	Fila del premio, Canjear, mantener.
Recuperar un código ya generado	3 toques	Avatar, Mis canjes, Mostrar.
Fijar una meta	2 toques	Premio, Fijar como meta.

6.4. Pantallas principales

Diseñamos 27 pantallas de 360 × 800 dp. Estas son las del caso de uso principal. El número de cada una coincide con el nombre del frame en Figma.



08 · Inicio 09 · Break validado 13 · Explorar 17 · Detalle

19 · Confirmar 20 · Código 21 · Canje pendiente 23 · Actividad

Figura 2: Pantallas del caso de uso principal.

Componentes y jerarquía

- **Tarjeta de saldo**: es lo primero que se ve. El número grande en lima responde "¿cuánto tengo?". El anillo muestra el avance del día contra la meta diaria.
- **Tarjeta de meta**: responde "¿cuánto me falta?". Cuando alcanza, aparece ahí mismo el botón Canjear (pantalla 09).
- **Fila de premio**: ícono, nombre, comercio, distancia y un chip con los puntos. El chip es lima si alcanza y gris si no.
- **Hoja inferior de Explorar**: el mapa orienta, pero la lista (abajo, al alcance del pulgar) es

la que se toca.

Estados

La consigna pide representar carga, contenido, vacío, error y offline. La tabla indica qué frame cubre cada caso.

Pantalla	Carga	Contenido	Vacío	Error	Offline
Primer uso	01	02 a 06		07 (permiso rechazado)	
Inicio	11	08, 09	10	No aplica, datos locales	12
Explorar	Igual que 11	13		14 (sin ubicación), 16	15
Canje	19 (mantener)	17, 18, 20		22 (sin stock)	21
Actividad	Igual que 11	23	24	No aplica, datos locales	23
Perfil		25, 26			26 (pendiente)

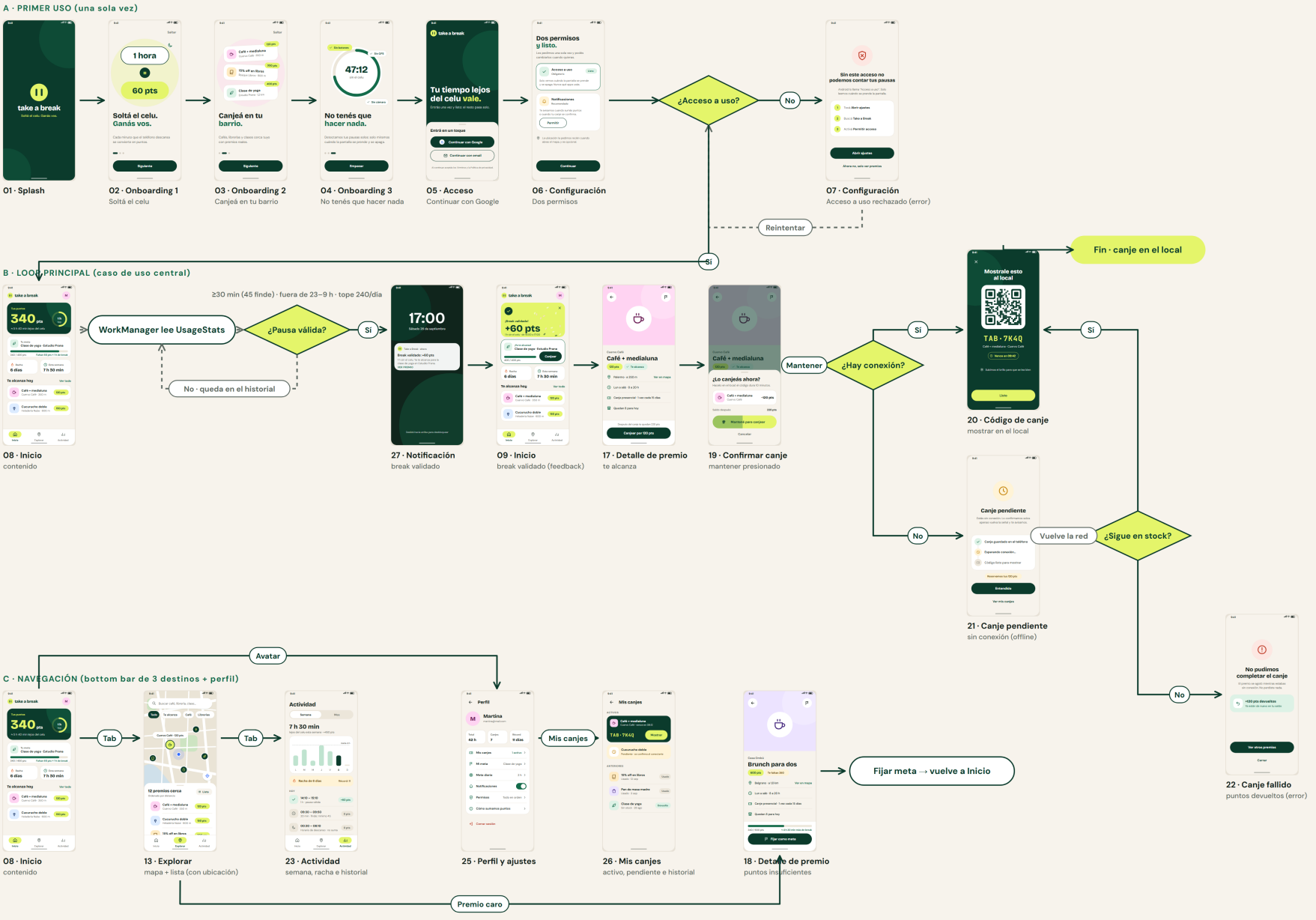
6.5. Flujo de pantallas

El diagrama de la página siguiente muestra el punto de entrada, las decisiones y los finales posibles:

- **Primer uso (A).** Splash, onboarding, acceso y permisos. La decisión es si el usuario da el acceso a uso. Si no lo da, puede reintentar o entrar igual a ver premios.
- **Ciclo principal (B).** Las flechas punteadas son transiciones del sistema. WorkManager lee los eventos, valida la pausa y, si es válida, notifica. Desde ahí el usuario llega al premio, confirma y, según haya conexión, obtiene el código o queda pendiente. El recorrido termina con el canje en el local o con los puntos devueltos.
- **Navegación (C).** Tres pestañas más el perfil desde el avatar.

Flujo principal · Take a Break

Punto de entrada → detección pasiva → validación → premio → canje. Las flechas punteadas son transiciones del sistema (sin acción del usuario).



7. Testear: plan de validación

Todavía no probamos el diseño con usuarios. Antes de empezar a programar vamos a hacer dos validaciones.

1. Prueba del prototipo. Con cinco personas del perfil de la sección 2 y el prototipo navegable de Figma. Primero una entrevista corta sobre cómo intentaron usar menos el celular. Después, cuatro tareas:

T1. Entender cómo se ganan puntos sin que nadie lo explique.

T2. Encontrar un premio que ya alcance y canjearlo.

T3. Decir por qué una pausa de anoche no sumó.

T4. Canjear sin conexión y explicar qué pasó.

Vamos a medir si completan cada tarea, el tiempo, la cantidad de toques y los errores, y al final aplicar el cuestionario SUS. Si dos de cinco personas no pueden completar una tarea, esa parte se rediseña antes de implementarla.

2. Prueba técnica de la detección. Una app mínima que lea los eventos de bloqueo con `UsageStatsManager` en tres dispositivos de marcas distintas y en el emulador. Sirve para confirmar que los eventos llegan completos antes de apoyar todo el producto en ellos.

8. Arquitectura

La app sigue MVVM con los principios de Clean Architecture. Las dependencias apuntan hacia el dominio: la UI conoce al ViewModel, el ViewModel a los casos de uso y los casos de uso solo conocen interfaces de repositorio. El dominio no importa nada de Android, así que las reglas de validación se prueban con tests unitarios comunes.

8.1. Capas y responsabilidades

Capa	Qué contiene
Presentación	Pantallas en Jetpack Compose y un ViewModel por pantalla. Cada ViewModel expone un `StateFlow<UiState>` con los estados *Loading*, *Content*, *Empty* y *Error*, más una marca de offline. La UI no decide nada: dibuja el estado y envía eventos.
Dominio	Modelos (`BreakSession`, `Reward`, `Business`, `Redemption`), las reglas de validación (`BreakRules`), los casos de uso (`SyncBreaks`, `GetHomeSummary`, `GetRewards`, `RedeemReward`, `SetGoal`, `GetBreakHistory`, `GetRedemptions`) y las interfaces de los repositorios.
Datos	Implementaciones de los repositorios, DAOs de Room, servicio Retrofit, DataStore y mappers entre entidades, DTOs y modelos.
Plataforma	Workers de WorkManager, lectura de eventos con `UsageStatsManager`, notificaciones y ubicación. Están detrás de interfaces para poder reemplazarlos en los tests.

```
com.takeabreak
 |- ui/ screens, components, theme, navigation
 |- presentation/ viewmodels y UiState
 |- domain/ model, rules, usecase, repository (interfaces)
 |- data/ local (Room, DataStore), remote (Retrofit), repository, mapper
 |- platform/ workers, usage (UsageStatsManager), notifications, location
 |- di/ AppContainer (inyección manual)
```

La inyección de dependencias es manual, con un `AppContainer`. Para el tamaño del proyecto alcanza y evita sumar Hilt.

8.2. Flujo de datos

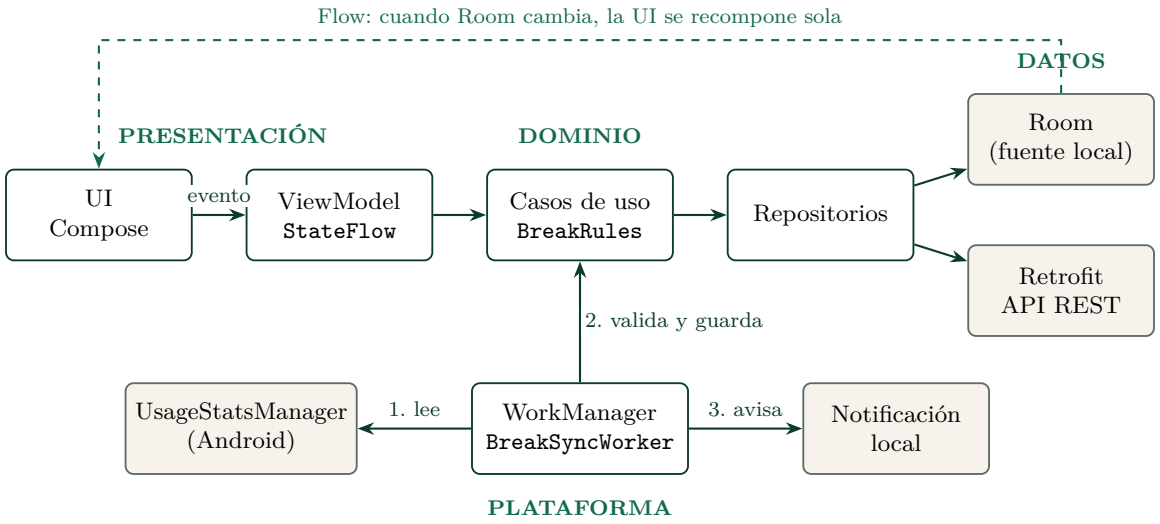


Figura 4: Flujo de datos. La línea punteada es el camino de vuelta reactivo.

Room es la única fuente de verdad. La UI nunca lee de la red: la red actualiza Room y Room avisa a la UI por `Flow`. Por eso la app se ve igual con o sin conexión.

8.3. Cómo se detecta una pausa

1. `BreakSyncWorker` corre cada 15 minutos (el mínimo de WorkManager) y también al abrir la app.
2. Pide a `UsageStatsManager` los eventos desde el último *checkpoint* guardado en DataStore. Solo usa cuatro tipos: pantalla encendida, pantalla apagada, pantalla bloqueada y desbloqueada.
3. Arma las pausas cerradas: desde que se bloquea hasta que se desbloquea. Si la pantalla se enciende por una notificación y se vuelve a apagar sin desbloquear, la pausa sigue.
4. `BreakRules` calcula los puntos, las pausas se guardan en Room, después se actualiza el checkpoint en DataStore y, si sumó, se muestra la notificación.

Cada pausa se identifica por la hora en que empezó, que es única en la tabla. Si el proceso muere antes de actualizar el checkpoint, la próxima lectura vuelve a traer esas pausas y Room las reemplaza en vez de duplicarlas, así que no se pierde ni se cuenta dos veces ninguna. Las reglas quedan en una función pura:

```
object BreakRules {
    const val DAILY_CAP = 240

    fun points(pause: Pause, pointsToday: Int): Int {
        val minMinutes = if (pause.isWeekend) 45 else 30
        val countable = pause.minutesOutside(restFrom = 23, restTo = 9)
        if (countable < minMinutes) return 0
        return countable.coerceAtMost(DAILY_CAP - pointsToday)
    }
}
```

8.4. Topología de infraestructura

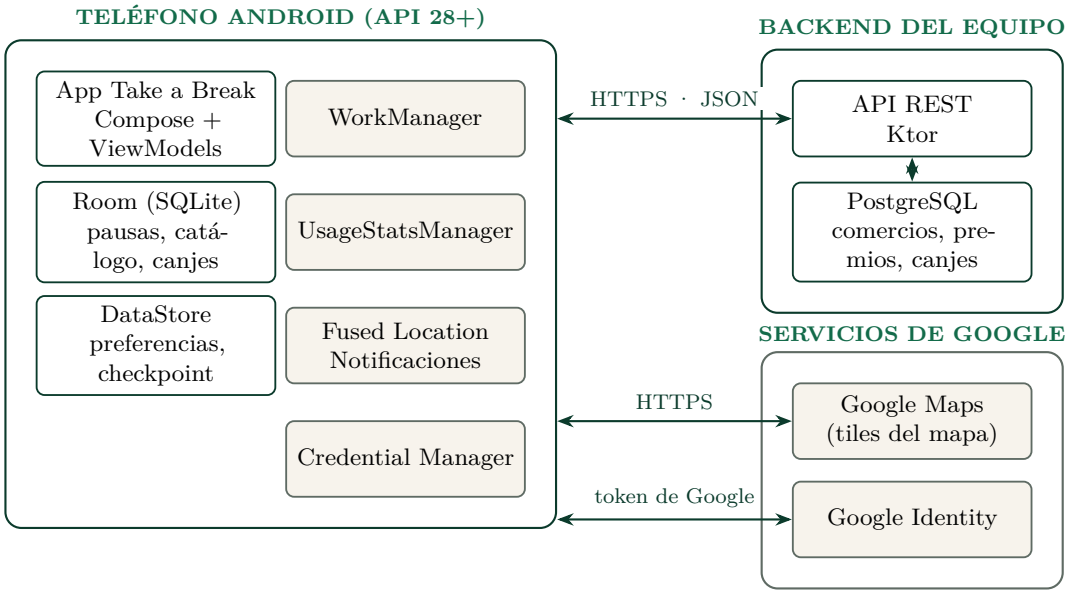


Figura 5: Topología. El teléfono funciona solo; la red se usa para el catálogo, los canjes, el mapa y el acceso.

El backend es una API REST propia, hecha en Ktor para usar el mismo lenguaje que la app, con PostgreSQL y desplegada en un servicio gratuito como Render. Expone pocos endpoints:

Endpoint	Uso
`POST /auth/google`	Recibe el token de Google y devuelve un token de sesión.
`POST /auth/email`	Registra o identifica al usuario con email y contraseña y devuelve un token de sesión.
`GET /rewards?since=`	Catálogo de comercios y premios. Con `since` devuelve solo lo que cambió.
`POST /points/daily`	Recibe el total de puntos por día, sin pausas ni horarios, y devuelve el saldo. Rechaza más de 240 puntos, días futuros o de más de 10 días.
`POST /redemptions`	Crea un canje y descuenta stock y saldo en una transacción. El UUID del teléfono evita duplicados en los reintentos.
`POST /redemptions/{id}/use`	Confirma la entrega con el PIN del comercio. Pasa el canje a usado con un cambio atómico y admite 5 intentos.
`GET /redemptions/{id}`	Estado de un canje (activo, usado, vencido, rechazado).

9. Persistencia y datos

Hay dos tipos de datos. Las pausas se generan en el teléfono y los canjes los confirma el servidor. Los comercios y los premios vienen del backend y se guardan en Room como caché.

Entidad	Campos principales	Origen	Dónde vive
`BreakSession`	id, inicio, fin, minutos válidos, puntos, estado (válida, corta, descanso, tope)	Teléfono	Room
`Business`	id, nombre, dirección, barrio, lat, lng, horario	API	Room (caché)
`Reward`	id, businessId, título, puntos, stock diario, actualizado	API	Room (caché)
`Redemption`	id (UUID), rewardId, puntos, código, estado (activo, usado, vencido, devuelto), creado, vence	Teléfono y API	Room y servidor

Relaciones. Un comercio tiene muchos premios y un premio puede tener muchos canjes. Las pausas no se relacionan con nada.

El saldo lo define el servidor. Lleva un libro de movimientos por usuario y el saldo es su suma, sin guardarse aparte. El teléfono guarda el último saldo confirmado y le suma los puntos de los días que todavía no informó, marcados como "por confirmar".

DataStore guarda lo que no es una tabla: el checkpoint de eventos, la meta elegida, la meta diaria, si las notificaciones están activas, la hora de la última sincronización, el último saldo confirmado, los días ya informados y el token de sesión cifrado.

Las pausas no se suben al servidor: solo se informa el total de puntos de cada día. Además de ahorrar red, es una decisión de privacidad: el horario en que alguien deja el teléfono dice mucho de su rutina.

10. Estrategia Offline First

La detección de pausas no usa la red en ningún momento. Lo único que depende de la conexión es el catálogo, el mapa, el envío de los puntos y los canjes.

Situación	Qué hace la app	Qué ve el usuario
Tiene conexión	Actualiza el catálogo cada 6 h, y al abrir Explorar si pasó más de 1 h. Informa los puntos de cada día que cambió y guarda el saldo.	Todo normal.
Pierde conexión	Sigue detectando y sumando. No se puede canjear y no se descuentan puntos.	Un aviso: "Sin conexión · tus pausas se siguen contando" (frames 12 y 15). El botón de canjear explica que hace falta conexión.
Recupera conexión	Un worker con la restricción de red informa los puntos, con reintentos y espera exponencial.	El saldo deja de figurar "por confirmar".
Datos viejos	Muestra el catálogo guardado. El stock se vuelve a validar en el servidor al canjear.	La hora de la última actualización.
Sin datos todavía	Solo puede pasar en Explorar la primera vez sin red. Inicio funciona igual.	Estado de error con Reintentar (frame 16).

Conflictos. Para el catálogo, el stock y el saldo manda el servidor. Si al canjear el premio se agotó, se dio de baja o no alcanza el saldo, no se crea el canje y no se descuentan puntos. Si un código vence sin usarse o se rechaza después de cinco PIN incorrectos, los puntos vuelven al saldo. El UUID que genera el teléfono evita duplicados si un envío se reintenta.

Mapa sin red. Los marcadores salen de Room, pero el fondo del mapa necesita conexión. En ese caso se muestra solo la lista.

11. Tecnologías previstas

Tecnología	Para qué la usamos
Kotlin y Jetpack Compose	Stack de la materia. La UI declarativa simplifica dibujar los distintos estados de cada pantalla.
Material 3	Componentes base sobre los que aplicamos nuestros colores y tipografías.
Navigation Compose	Tres pestañas, detalle, canje y perfil. Permite abrir el premio desde la notificación (deep link).
ViewModel, StateFlow y Coroutines	Estado que sobrevive a la rotación y trabajo asíncrono sin bloquear la UI.
Room	Fuente de verdad local: pausas, catálogo en caché y canjes. Es la base del offline first.
DataStore	Preferencias, checkpoint de eventos y token de sesión.
Retrofit, OkHttp y kotlinx.serialization	Comunicación con la API del equipo.
WorkManager	Lectura periódica de eventos, sincronización del catálogo y envío de los puntos. Sobrevive a reinicios del teléfono.
UsageStatsManager	Fuente de la detección. Requiere el permiso especial de acceso a uso, que el usuario da desde Ajustes.
Notificaciones (NotificationCompat)	Aviso de pausa validada. En Android 13 o superior se pide el permiso.
Maps Compose y Fused Location	Mapa de comercios y ubicación puntual, solo en primer plano.
Credential Manager	Acceso con Google en un toque.
JUnit, Turbine y Compose UI Test	Tests de reglas, ViewModels y pantallas.
Ktor y PostgreSQL (backend)	API propia en el mismo lenguaje, con datos semilla de comercios y premios.

Permisos que pide la app: acceso a uso (obligatorio para sumar puntos), notificaciones (recomendado) y ubicación aproximada o precisa mientras se usa la app (opcional). No pide ubicación en segundo plano, cámara ni contactos.

12. Repositorio y equipo

12.1. Repositorios

Código de la app	`https://github.com/DesarrolloApps1/tpo-da1`
Documentación y diseño	`https://github.com/DesarrolloApps1/docs` (este documento y los archivos de Figma)
Diseño en Figma	`https://www.figma.com/community/file/1687633708033076651/take-a-break-diseno`
Backend	`https://github.com/DesarrolloApps1/tpo-da1-backend` (API REST en Ktor)

12.2. Forma de trabajo

Los tres repositorios siguen el mismo esquema de ramas.

- **Ramas fijas.** Cada repositorio tiene dos ramas permanentes. `main` guarda la versión estable, la que se puede entregar. `dev` es donde se va integrando el trabajo iterativo.
- **Ramas de trabajo.** Para agregar algo nuevo, cada integrante crea una rama a partir de `dev` con el formato `nombre/feature`. Por ejemplo, si Joaquín trabaja en mejoras de la interfaz, la rama se llama `joa/mejoras-ui`.
- **Pull requests.** Cuando el trabajo en la rama está listo, se abre un pull request hacia `dev`. Una vez que `dev` acumula avances estables suficientes, se abre un pull request de `dev` a `main`.
- **Limpieza.** Después de que una rama se mergea en `dev`, se elimina, para no acumular ramas sin uso.
- **Commits.** Chicos y frecuentes, en español y con prefijo: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`.
- **Tareas.** Un issue por requisito o pantalla, organizados en un tablero de GitHub Projects.



Figura 6: Flujo de ramas en los repositorios del equipo.

12.3. Integrantes, roles y responsabilidades

Los roles no son exclusivos. Todos revisan código de otras áreas y tienen que poder explicar la app completa.

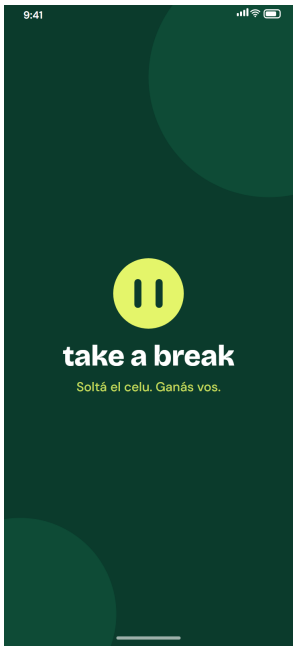
Integrante	Rol principal	Responsabilidades	Participa en
Velazquez, Joaquin Eduardo	Arquitectura, integración y documentación	Estructura de capas, navegación, AppContainer, revisión de PRs, README y este documento	Todas las áreas
Maci, Abril	UI y UX	Diseño en Figma, tema de Compose, pantallas y estados, accesibilidad, pruebas con usuarios, tests de UI	Todas las áreas
Jerez, Juan	Datos y sincronización	Room, DataStore, Retrofit, backend y estrategia offline	Todas las áreas
Papa, Valentin	Detección, plataforma y testing	Worker, UsageStatsManager, reglas de validación, notificaciones, tests del dominio y ViewModels	Todas las áreas

Cambios en el equipo: Ninguno hasta la fecha.

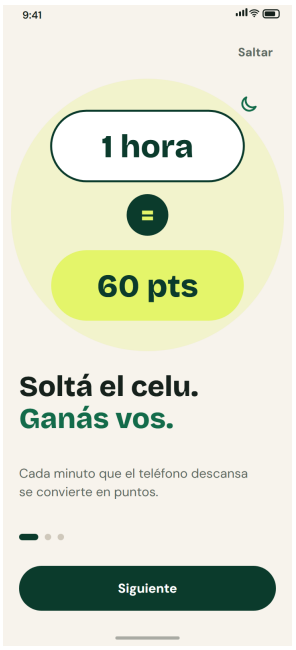
A. Todas las pantallas

Cada frame se nombra con su número. Los archivos editables están en `docs/figma/screens`.

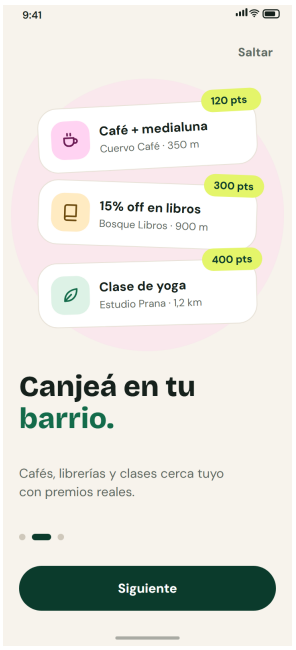
A · Primer uso



01 · Splash



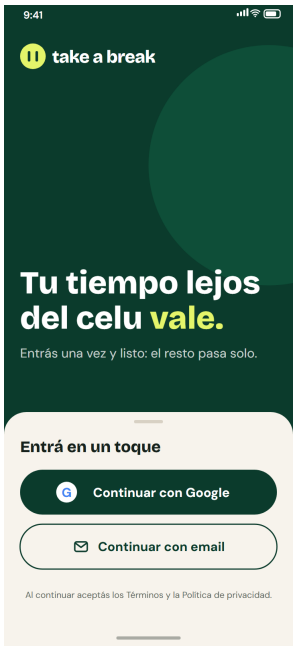
02 · Onboarding 1



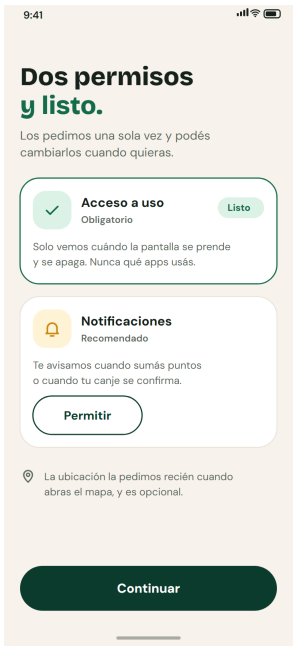
03 · Onboarding 2



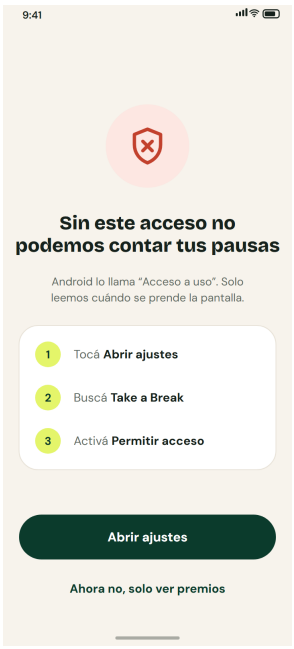
04 · Onboarding 3



05 · Acceso

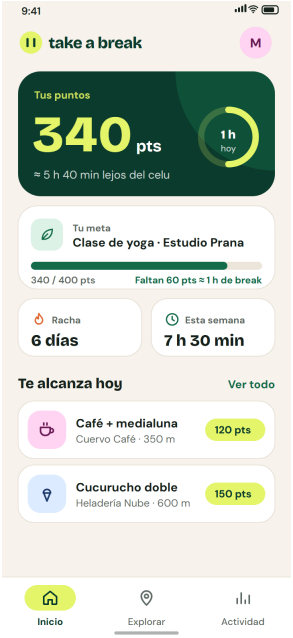


06 · Permisos



07 · Permiso rechazado

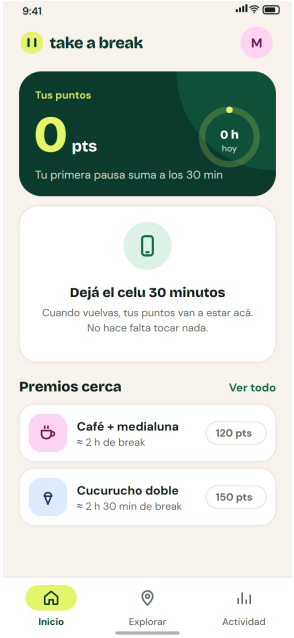
B · Inicio



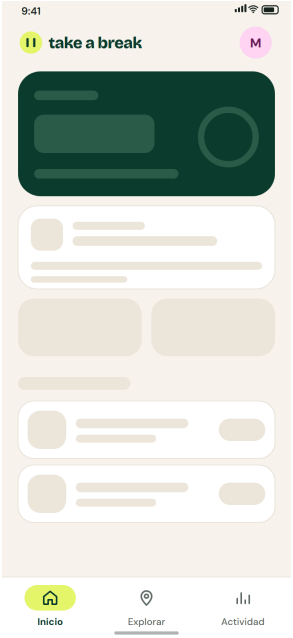
08 · Contenido



09 · Break validado



10 · Vacío

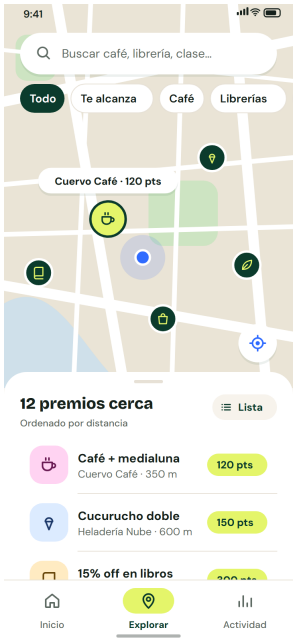


11 · Cargando



12 · Sin conexión

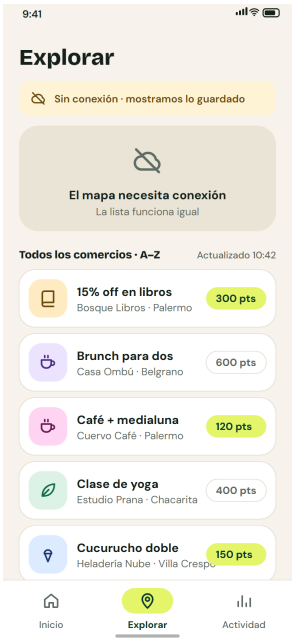
C · Explorar



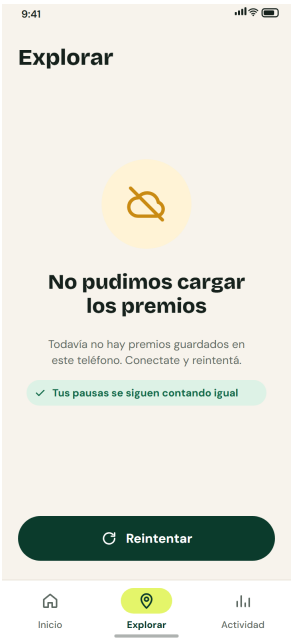
13 · Mapa y lista



14 · Sin ubicación



15 · Sin conexión



16 · Error sin datos

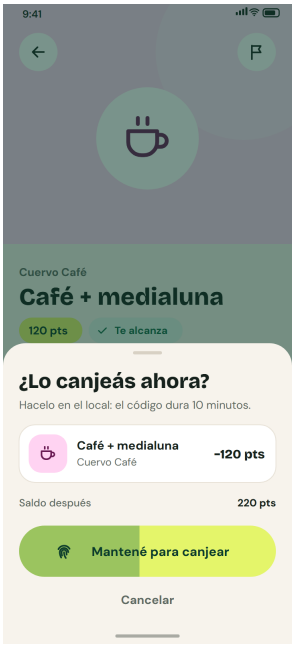
D · Canje



17 · Detalle



18 · No alcanza



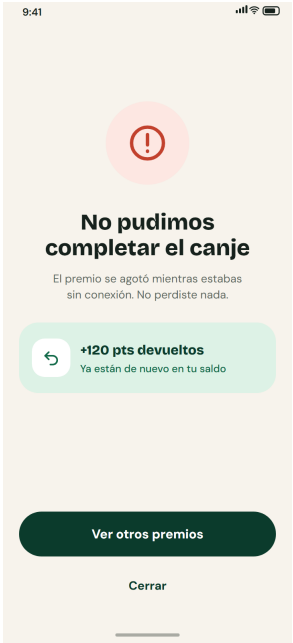
19 · Confirmar



20 · Código



21 · Pendiente



22 · Fallido

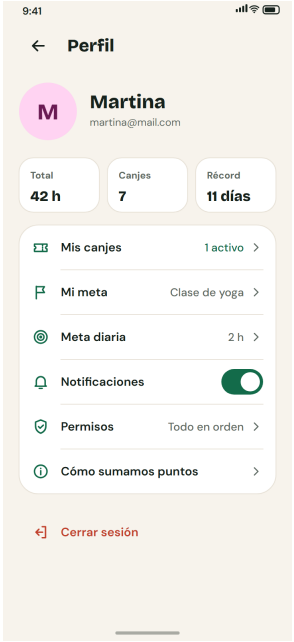
E · Actividad, F · Perfil y G · Sistema



23 · Actividad



24 · Actividad vacía



25 · Perfil

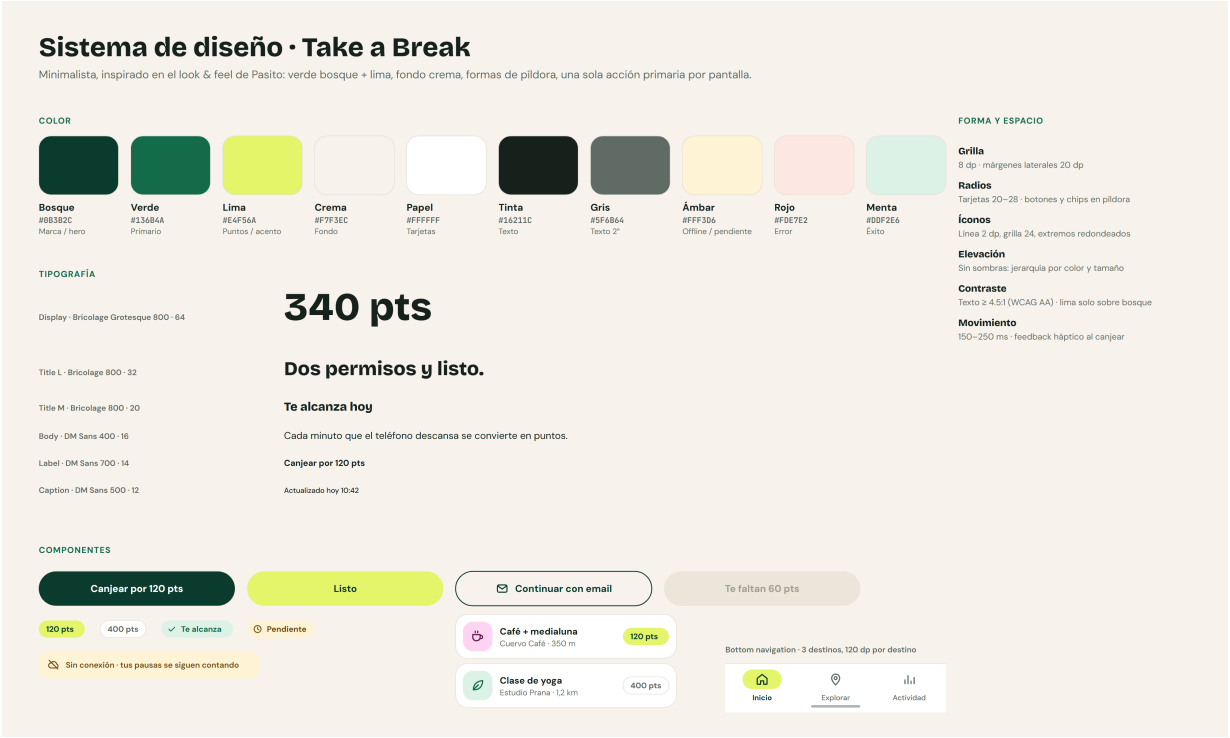


26 · Mis canjes



27 · Notificación

B. Sistema de diseño



C. Bocetos originales

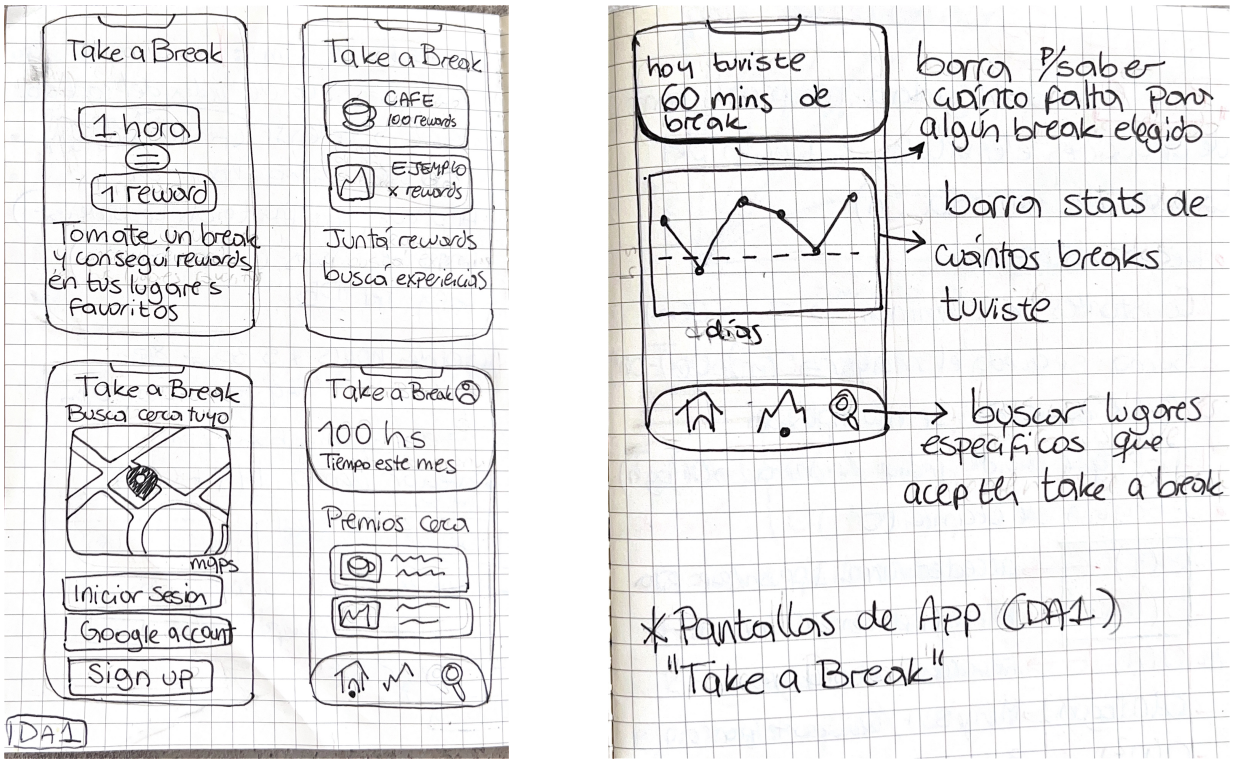


Figura 7: Primeros bocetos en papel.